

AKKA

Reliable AI for Every Industry
The Agentic Systems Platform

Akka vs. CrewAI

A comparison for teams building agentic AI

July 2026

CrewAI gets you to a multi-agent demo fast; with Akka the production system — durability, availability, and governance — is owned by the platform, not by you. CrewAI is an ergonomic Python framework for prototyping role-based crews and event-driven flows. It does not provide a durable execution runtime, active-active HA/DR, a numeric availability SLA, or embedded governance — those you build, integrate, and operate yourself. Akka delivers them as one runtime, under contract — and Akka Specify can deliver and run the entire governed system for you as a guaranteed outcome.

99.9999%

AKKA AVAILABILITY SLA

Weeks

AKKA SPECIFY DELIVERY

Fixed

ONE ALL-IN PRICE

90%

LESS INFRASTRUCTURE

DIMENSION	CREWAI	AKKA
What it is	A Python multi-agent framework (Crews + Flows), plus the AMP control plane	A full-stack agentic systems platform
Scope	Agent authoring and orchestration; durable runtime, HA/DR, streaming, and governance are sourced, integrated, and operated by the customer	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
How you reach production	Self-build, or contract a labor-led integration engagement	Build with spec-driven development, or Akka Specify delivers and runs it
Commercial model	Execution-metered, plus separately provisioned infrastructure and build-and-operate labor	One fixed price — platform, infrastructure, tokens, training, delivery, and operations
Outcome guarantee	Effort only; the outcome is not guaranteed	The delivered outcome is guaranteed
Durable execution	@persist() saves flow state; crash recovery is not built in — failures are detected and replayed manually; the inner agent loop is not persisted	Event-sourced durable state; runtime handles recovery automatically
Availability SLA	None published	99.9999% — entire platform, backed by indemnities
HA / DR	No published active-active or RTO/RPO posture	Active-active across regions; sub-1-minute RTO, zero-byte RPO
State / memory	Configurable memory; flow state persisted to a configured backend	Durable in-memory, 4ms reads / sub-10ms writes, PB cache, replayable
Real-time streaming	Not part of the framework	Built into the runtime — backpressured, petabyte-scale, no external broker
Governance / EU AI Act	Guardrails, tracing, audit logging; no inline EU AI Act enforcement, classification, or sealed artifact	Aspect-woven runtime enforcement + full pre-production governance
Cost model	Execution-metered (Enterprise: up to 30,000/mo, then \$0.50 each) + separate infra for the rest	Shared compute; up to 90% lower infrastructure for the same workload
Improves after go-live	Not provided	Continuous evaluation, reinforcement learning, and distillation — up to 80% lower token cost

		over time
Certifications	SOC 2, HIPAA, GDPR	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF)
Viability	Venture-funded: \$18M total, Series A led by Insight Partners (Oct 2024)	Profitable; Dell Technologies Capital largest shareholder; 18 years, 100,000+ deployments

1. CrewAI Is a Framework; Akka Is a Platform

CrewAI is a Python framework for building multi-agent systems — Crews of role-based autonomous agents and event-driven Flows — built from scratch, [independent of LangChain](#). It is fast and ergonomic to a working multi-agent demo, and that is its genuine strength. Production is a different problem: with CrewAI, the durable runtime, high availability, streaming, and governance around the agents are sourced, integrated, and operated by the customer.

CrewAI Enterprise / AMP (the [Agent Management Platform](#)) adds a control plane on top — a visual editor, an AI copilot, deployment to managed cloud or a private VPC, RBAC, SSO, tracing, and task guardrails. AMP makes the framework easier to operate. It does not change what the framework is: an agent-authoring layer, not a guaranteed runtime.

Capability	CrewAI	Akka
Multi-agent authoring (crews, roles, flows)	Yes	Yes
Durable execution runtime	Manual (@persist() + manual replay)	Built in, automatic
Active-active HA / DR	No published posture	Built in
Durable memory	Configurable, backend-dependent	Built in, 4ms / sub-10ms
Real-time streaming	Not in the framework	Built in, backpressured, petabyte-scale
Governance / policy enforcement	Guardrails + tracing only	Inline, runtime-embedded
Pre-production governance	None	Classification, sign-offs, sealed posture
Delivery & operation	The customer builds and operates the application, or contracts a labor-led integrator	Delivered and operated by Akka Specify as a guaranteed outcome, in weeks

2. Two Ways to Production: Build It, or Have It Delivered

Akka offers two ways onto one platform: build the system yourself with spec-driven development, or have **Akka Specify** deliver and operate the entire governed system for you as a guaranteed outcome — in weeks, for one fixed price. CrewAI offers neither a platform nor a delivery model: with CrewAI, the durable runtime, HA/DR, streaming, and governance around the agents are sourced, integrated, and operated by the customer, and a CrewAI application is the customer's to build, run, and own.

	With CrewAI	With Akka Specify
Model	Self-build, or a labor-led integration engagement	You provide the specifications
Who does the work	Your engineers, or forward-deployed / staff-augmentation contractors	Akka generates, governs, delivers, and runs the system
Timeline	Months, by construction	Weeks, not quarters
Billing	Execution-metered, plus separately provisioned infrastructure	One fixed price
Afterward	You inherit the system and its operations	Kept up, safe, and improving — operated by Akka
Guarantee	Effort is guaranteed; the outcome is not	The delivered outcome is guaranteed

With CrewAI, the durable runtime, availability, and governance around the agents are the customer's to build, integrate, and operate. A team that cannot self-build contracts a labor-led integration engagement — billed for time and materials, guaranteeing effort rather than the outcome, and handing back a system the team then owns and operates. Akka Specify inverts that: you provide a handful of plain-language specifications, and Akka generates, tests, governs, deploys, and runs the system — then keeps it available, safe, and improving under one agreement.

3. Reliability: Durable Execution, Availability, and Disaster Recovery

CrewAI does not publish a numeric availability SLA, an active-active HA/DR posture, or RTO/RPO targets. Durability is partial and manual: the [@persist\(\) decorator](#) saves flow state at chosen points, but crash recovery is not built in — the team detects the failure and triggers the

replay themselves, and the autonomous agent reasoning loop is not persisted, so a crash inside it cannot be resumed. [Diagrid](#) sells a Dapr-based product specifically to add "true durable execution" to CrewAI, which confirms the gap from the ecosystem's own side.

Metric	CrewAI	Akka
Availability SLA	None published	99.9999%
Allowed downtime / year	Not specified	~31 seconds
Crash recovery	Manual detect + manual replay; inner loop not persisted	Automatic, runtime-handled
RTO	No published target	Sub-1 minute
RPO	No published target	Zero byte
HA / DR	No published active-active posture	Active-active across regions
SLA scope	The customer owns application availability	The entire platform
Who operates it	The customer (or an inherited integrator)	Akka SREs, 24/7

Akka's reliability is structural: actor-based concurrency and clustering, durable sharded in-memory state replayable from its event journal, and brokerless backpressured streaming with a 1-minute RTO and 0-byte RPO. The platform owns and operates the SLA, with 24/7 SRE and contractual indemnities.

4. Cheaper to Operate — and It Keeps Getting Cheaper

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required for the same agentic transaction volume, not list price. The drivers are actor concurrency, shared compute, and micro-checkpointing: Akka processes ~10 trillion tokens per core per year versus ~2 trillion for comparable solutions, using ~80% less compute than Python-based frameworks. Manulife reported up to 300% more concurrency and 30–50% faster processing after porting Python-based systems to Akka. CrewAI is a Python framework billed by execution — the Enterprise tier includes up to 30,000 executions per month, with additional executions at \$0.50 each — and that meter covers agent runs only. The durable runtime, HA/DR, streaming, observability, and governance are separate infrastructure the customer provisions and pays for, plus the labor to build and operate it. Akka runs all of it on one shared-compute runtime for **one fixed price** finance can forecast, rather than per-execution metering that moves with load.

And the cost falls after go-live

The delivered outcome compounds. Akka Verify runs continuous evaluation, reinforcement learning on production and synthetic data, and distillation to smaller specialized models — cutting token cost **up to 80%** while raising accuracy over time. CrewAI's tracing and guardrails observe and log; they do not retrain, distill, or ship the improvement. With Akka Specify, that tuning is part of the operated outcome, not a project the customer runs later.

5. Governance and the EU AI Act

CrewAI provides task guardrails, tracing, audit logging, and RBAC, and holds SOC 2, HIPAA, and GDPR — security and operational controls. It does not provide inline EU AI Act enforcement: no real-time policy enforcement woven into the runtime, no decision explainability, no human pause/override of a running process, no hash-chained immutable interaction ledger, no pre-deployment classification, and no sealed audit artifact. Comprehensive EU AI Act conformance on CrewAI requires integrating a separate governance platform.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover
Incorrect information to authorities	EUR 7.5M or 1.5% global turnover

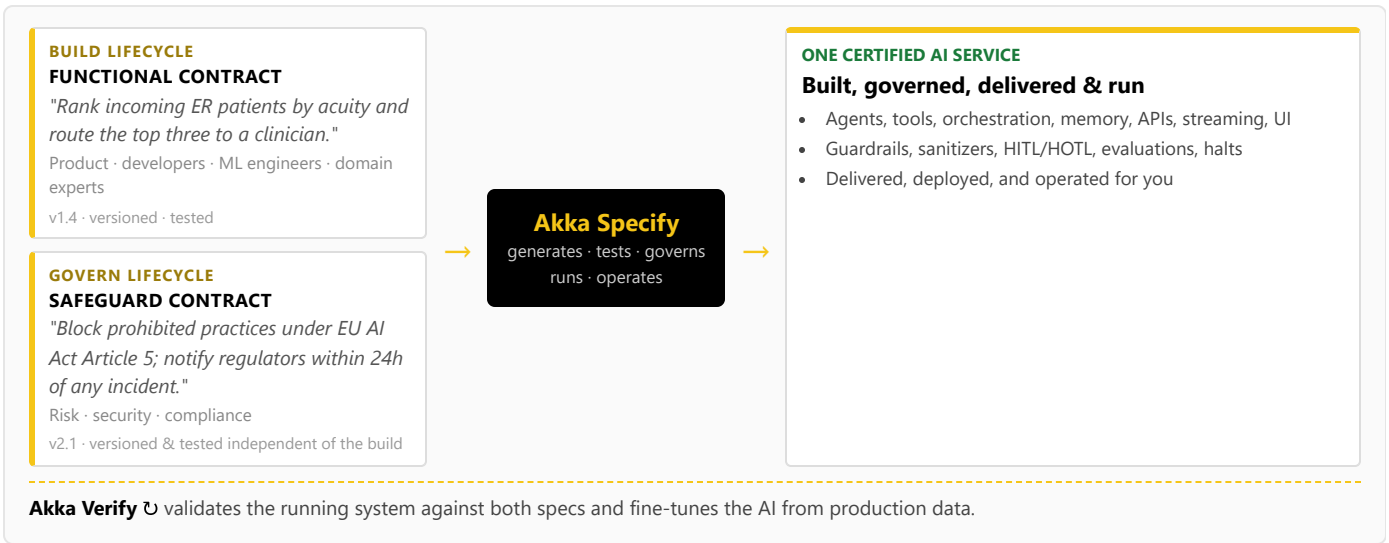
High-risk AI carries a 10-year logging-retention obligation (Art. 72).

How Akka governs

At the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; atomic PII scrub-with-explain; classification against 189 regulations and 962 controls — 574 controls carrying a financial penalty (across 89 regulations) — before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Governance CrewAI leaves to a bolt-on, Akka enforces inline — and delivers it already wired when the system is built through Akka Specify.

6. One Certified System — Built, Governed, Delivered, and Run

Building on CrewAI means engineers writing Python crews and flows; there is no built-in path for a risk officer or compliance owner to author and version a governance contract independently of the application, and no governance lifecycle in the framework. Akka runs two independent lifecycles on one platform via **Akka Specify**:



The build lifecycle and the governance lifecycle are versioned and tested independently, by different audiences. Akka generates, tests, governs, **runs, and operates** one certified AI service that satisfies both — and, through **Akka Specify**, the whole certified system is delivered and operated as a guaranteed outcome. CrewAI has no equivalent for either the independent governance lifecycle or the delivery model.

7. Real-Time Streaming at Petabyte Scale

CrewAI has no streaming engine; real-time pipelines are provisioned separately. Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

8. For the Buyer: Risk, Compliance, and Accountability

Buyer concern	CrewAI	Akka
Certifications & audits	SOC 2, HIPAA, GDPR	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Delivery & outcome	Self-build or a time-and-materials integrator; effort is guaranteed, the outcome is not	Akka Specify delivers and operates the system for one fixed price; the outcome is guaranteed
Scope of accountability	Agent framework + control plane; you integrate and operate the durable runtime, HA/DR, and governance	One platform, one SLA, 24/7 SRE — Akka owns the running system
Availability commitment	No published numeric SLA	99.9999%, backed by contractual indemnities
Risk transfer	Standard cloud terms	Availability and data-integrity guarantees backed by contractual indemnities
Track record & funding model	Venture-funded: \$18M total, Series A led by Insight Partners (Oct 2024); used across many Fortune 500 firms	Profitable and self-funding; 18 years and 100,000+ deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Per-execution metering that scales with load, plus build-and-operate labor	One fixed price finance can forecast

CrewAI has real enterprise traction — it reports use across a large share of the Fortune 500 and hundreds of millions of agent executions a month. The decision is not whether CrewAI works for prototyping; it is who owns production. With CrewAI the customer owns the durable runtime, the availability target, the DR plan, and the governance stack. With Akka the platform owns them, under contract — or, through Akka Specify, Akka delivers and operates the whole system for you.

Customers Running Agentic and Real-Time Systems on Akka

Manulife 2,000 developers across 100 projects on one governed platform	Tubi 5B tok/s real-time hyper-personalization engine	Swiggy 71ms order-assignment AI, ~50% latency reduction	John Deere 1,000+ tractor sensors turned into real-time insight	Verizon 750% order-processing capacity gain; 6s → 2.4s response
--	--	---	---	---

Common Questions

We don't have a team to build this — what are our options?

Two. Build it on the platform with spec-driven development, or have Akka Specify deliver and operate it for you. You provide plain-language specifications; Akka generates, governs, delivers, and runs the system as a guaranteed outcome, for one fixed price. With CrewAI, production is self-build or a labor-led integration engagement you then own.

How is Akka Specify different from hiring an integrator to build our CrewAI app?

An integration engagement sells effort — time-and-materials over months — and hands back a system you own and operate; the outcome is not guaranteed. Akka Specify sells the outcome: a governed system

Doesn't CrewAI's @persist() give us durable execution?

@persist() saves flow state to a configured backend, but crash recovery is not automatic — your team detects the failure and triggers the replay, and the agent's inner reasoning loop is not persisted, so a crash there cannot be resumed. Third parties such as Diagrid sell durable-execution add-ons specifically to fill this gap. Akka's runtime recovers automatically from an event journal with a sub-1-minute RTO and zero-byte RPO.

Can we add governance on top of CrewAI?

You can add guardrails, tracing, and audit logging, and integrate a separate governance platform. But the EU AI Act expects enforcement

delivered in weeks for one fixed price, then kept up, safe, and improving by Akka. You own the specifications, not the operational burden.

We already prototype with CrewAI. Why move to Akka?

CrewAI is good for getting a multi-agent system working quickly. The gap appears in production: durable execution is manual, there is no published availability SLA or active-active HA/DR, and EU AI Act governance is not enforced in the runtime. Akka provides all of that as one platform, so you do not assemble and operate it yourself. You can keep prototyping in CrewAI and standardize production on Akka.

Sources

CrewAI OSS framework: github.com/crewAIInc/crewAI — Python framework for role-based crews and event-driven flows, built from scratch, independent of LangChain

CrewAI AMP / Enterprise: docs.crewai.com/en/enterprise/introduction — control plane: visual editor, AI copilot, cloud or private VPC deploy, RBAC, SSO (Entra/Okta), tracing, task guardrails

CrewAI durable-execution gap: speakeasy.com/blog/ai-agent-framework-comparison; diagrid.io/solutions/crewai-production — @persist() saves flow state; crash recovery not built in; manual detect + replay; inner loop not persisted; Diagrid adds true durable execution via Dapr

CrewAI compliance: github.com/crewAIInc/crewAI/discussions/2404 — SOC 2, HIPAA, GDPR, RBAC, audit logging (no published availability SLA, active-active HA/DR, or EU AI Act enforcement found)

CrewAI pricing: techjacksolutions.com/ai-tools/crewai/crewai-pricing; toolworthy.ai/tool/crewai — execution-metered; Enterprise up to 30,000 executions/mo, additional at \$0.50 each

CrewAI funding & traction: siliconangle.com/2024/10/22/insightpartners.com

inline to the runtime: immutable records witnessed as they happen, human override of running processes, pre-deployment classification, and a sealed audit artifact. Bolt-on tools observe after the fact and cannot gate a deployment. Akka embeds all of this and covers pre-deployment governance.

Is CrewAI cheaper because the framework is open source?

The core framework is open source, but production CrewAI means CrewAI Enterprise / AMP (execution-metered) or self-hosting — where you build and operate the durable runtime, HA/DR, streaming, observability, and governance yourself. Akka's shared-compute model is up to 90% cheaper to operate than the equivalent assembled stack, on one fixed price.

— \$18M total, Series A led by Insight Partners, Oct 2024; used across a large share of the Fortune 500; ~450M agents/month (Jan 2026)

Akka Specify (spec-driven delivery): akka.io / akka.io/llms.txt — you provide the specifications; Akka generates, tests, governs, delivers, and operates the system as a guaranteed outcome; one fixed price covering platform, infrastructure, tokens, training, delivery, and operations; delivered in weeks; continuous improvement via reinforcement learning and distillation (up to 80% lower token cost, higher accuracy)

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 189 regulations / 962 controls / 574 controls carrying a financial penalty (across 89 regulations); 100,000+ deployments / 18 years; profitable; Dell Technologies Capital